A Bite of Peer-to-Peer

Khoa A. Vu & Kelly Pham

Student ID: 030063200 & 03039600

California State University, Long Beach

CECS 327 - Section 01

Dr. Hailu Xu

October 31, 2025

Objective

The purpose of this project is to get introduced to distributed systems by developing a peer-to-peer network where we:

1. Implement a P2P network architecture with a bootstrap node.
2. Develop a peer registration and messaging system

Implementation

To begin our P2P server, we initialized a Flask app through app = Flask(_name__) where we utilized uuid 4 to get a unique identifier of the node. In our main function we are making sure that all other nodes can connect where the current node is 0.0.0.0 listening on port 5000. For any nodes that are discovered, they will be added to the initial empty set through the line of code 'peers = set()'. Once peers are discovered, each node can send and receive messages using the unique URL of the target node, which acts as its identifier within the network. In both p2p_node.py bootstrap.py, we made sure to include a register function where we show the successful registration status of the node and an error message in the case of failure. For this lab, it was important to include the methods GET and POST. GET allows any node to receive the set of the discovered nodes and with that, the bootstrap server is allowed to respond with JSON format with the peered URLs. POST makes it so that a node is able to register itself in the peer set with the URL so that they can receive any messages, allowing for peer registration. To make sure that our initial target node is not added into the peer list, we first check with peer != my_url before adding it into the set. In the message function and by including /message, it allows nodes to receive messages from other peers. As seen in the output, the message includes a sender ID and content, which is printed logging and verification. With this design, we are able to connect

multiple nodes such as ten or twenty without being limited to only two or three nodes. When executing the test cases, we did run into some trouble where node1 was already in use by the container but it was a simple fix of resetting the container and removing the node to build it again. Additionally, we had to include the environment variables node_url and bootstrap_url to make sure that the containers know how to reach each other and be able to recognize each other by their unique names and identifiers.

Screenshots

bootstrap.Dockerfile

Project 3 > p2p-system > bootstrap.Dockerfile > ...

```
FROM python:3.9-slim

# Sets up the application directory inside the container.
WORKDIR /app

# Copies your bootstrap server code into /app.
COPY bootstrap.py .

# Installs Flask — the only required dependency for your bootstrap node.
RUN pip install flask

# Optional but recommended) documents that your app listens on port 5000.
# Doesn't actually open the port, but helps when linking containers via Docker Compose or networks.
EXPOSE 5000

# Tells Docker to start your bootstrap service automatically when the container runs.
CMD ["python", "bootstrap.py"]
```

bootstrap.Dockerfile - Defines how to containerize the bootstrap node.

bootstrap.py 1

Project 3 > p2p-system > bootstrap.py > get_peers

```python
# Imports Flask for web server functionality, request for incoming POST data, and jsonify for formatting JSON responses.
from flask import Flask, request, jsonify

app = Flask(__name__) # Initializes the Flask app.
registered_peers = set() # registered_peers is a Python set() to store unique peer URLs (avoiding duplicates).

# Root endpoint to confirm the bootstrap node is running.
@app.route('/')
def index():
    return jsonify({"message": "Bootstrap node is running!"}) # Returns JSON text message.

# Handles registration of peers.
@app.route('/register', methods=['POST'])
def register():
    peer = request.json.get("peer")
    if peer:
        registered_peers.add(peer) # Adds valid peers to the registered_peers set.
        return jsonify({"status": "registered", "peer": list(registered_peers)}) # Returns a success JSON if added
    return jsonify({"error": "No peer provided"}), 400 # or return error code 400 if "peer" missing.

# Converts set → list because JSON doesn't support sets.
@app.route('/peers', methods=['GET']) #GET info from reg. peers
def get_peers():
    return jsonify({"peers": list(registered_peers)}) # Returns the list of all registered peers.

if __name__ == '__main__': # Runs the Flask app, listening on port 5000 and all network interfaces.
    app.run(host='0.0.0.0', port=5000) # In Docker, 0.0.0.0 is necessary so the service is reachable from other containers.
```

bootstrap.py - Acts as a central registry. Keeps a list of all active peers that have registered.

Dockerfile

Project 3 > p2p-system > Dockerfile > ...

```dockerfile
# Base image containing Python.
FROM python:3.9-slim

# Set working directory.
WORKDIR /app

# copy your script inside the container.
COPY p2p_node.py .

# install Flask dependency.
RUN pip install flask requests

EXPOSE 5000

# start the Python app when the container runs.
ENTRYPOINT ["python", "p2p_node.py"]
```

Dockerfile - Builds the Docker image for all peer nodes.

p2p_node.py 1 ×

Project 3 > p2p-system > p2p_node.py > ...

```python
'''
Citation(s):
1) “Generating Random id's using UUID in Python.” GeeksforGeeks, 04 Apr 2025.
2) “How to Build a Flask Python Web Application from Scratch.” DigitalOcean, updated December 12 2024.
3) “Flask HTTP methods, handle GET & POST requests.” GeeksforGeeks, Last Updated 23 Jul 2025.
'''

import sys # read command-line arguments (port).
import threading # used later for periodic peer refreshes (you prepared for future use).
import time # used later for periodic peer refreshes (you prepared for future use).
import requests # to send HTTP requests to bootstrap or peers.
import uuid # to generate a unique node ID.
import os
from flask import Flask, request, jsonify # to create a lightweight web API.

app = Flask(__name__)
node_id = str(uuid.uuid4()) # uniquely identifies the node.
peers = set() # stores known peer URLs.

bootstrap_url = os.getenv("BOOTSTRAP_URL", "http://host.docker.internal:5000")  # points to the bootstrap node on the host machine (important for Docker).
port = int(sys.argv[1]) if len(sys.argv) > 1 else 5000 # can be overridden via command-line, defaulting to 5000.
my_url = os.getenv("NODE_URL", f"http://host.docker.internal:{port}")

# register with bootstrap node
def register_with_bootstrap(port):
    """
    Builds the node's own address (e.g., http://localhost:5001).
    Sends a POST request to bootstrap's /register.
    Logs success or failure.
    flush=True ensures immediate console output (important inside Docker).
    """
    try:
        res = requests.post(f"{bootstrap_url}/register", json={"peer": my_url})
        if res.ok:
            data = res.json()
            peers.update(data.get("peers", []))
            print(f"[OK] Registered as {my_url}, peers: {peers}", flush=True)
        else:
            print(f"[ERR] Bootstrap returned {res.status_code}", flush=True)
    except Exception as e:
        print(f"[ERR] Registration failed: {e}", flush=True)
```

```python
#discover peers from bootstrap
def discover_peers():
    """
    Gets the /peers list from the bootstrap node.
    Adds all peers except itself to the local peers set.
    Logs the list of peers found.
    """
    try:
        res = requests.get(f"{bootstrap_url}/peers")
        if res.ok:
            peer_list = res.json().get("peers", [])
            for peer in peer_list:
                if peer != my_url:
                    peers.add(peer)
            print(f"[INFO] Discovered peers: {peers}", flush=True)
    except Exception as e:
        print(f"[ERR] Discovery failed: {e}", flush=True)

def continuous_discovery(interval=10):
    while True:
        discover_peers()
        time.sleep(interval)

# receive and send message
@app.route('/')
def index(): # Health check endpoint showing node's unique ID.
    return jsonify({"message": f"Node {node_id} is running!"})

@app.route('/message', methods=['POST'])
def message():
    """
    receives messages from other nodes and logs them
    logs the message sender and contents
    returns confirmation JSON { "status": "received" }.
    """
    data = request.json
    sender = data.get("sender")
    msg = data.get("msg")
    print(f"Received message from {sender}: {msg}", flush=True)
    return jsonify({"status": "received"})
```

```python
#get peer list when requested
@app.route('/peers', methods=['GET'])
def get_peers():
    return jsonify({"peers": list(peers)}) # Converts set → list for JSON serialization.

def send_message_to_peers(msg):
    for peer in list(peers):
        try:
            requests.post(f"{peer}/message", json={"sender": my_url, "msg": msg})
        except Exception as e:
            print(f"[WARN] Could not send to {peer}: {e}", flush=True)

if __name__ == '__main__':
    """
    First registers with the bootstrap node.
    Then fetches the current peer list.
    Starts Flask server accessible externally on port 5000 (inside the container).
    """
    register_with_bootstrap(port)
    threading.Thread(target=continuous_discovery, daemon=True).start()
    app.run(host='0.0.0.0', port=port)
    #start flask on all interfaces 0.0.0.0
```

p2p_node.py - The main logic for each peer — registers itself, discovers others, and sends/receives messages.

Contributions

- Kelly Pham: Worked on and created the baseline of the files needed for the project, and writing up the project report
- Khoa Vu: Finalized/tested the multiple files, Docker Compose setup, while also creating the README and video recording.
- Both: Reviewing task, documentation.

Result

Execution Demonstration Video

References

1) Dyouri, Abdelhadi. “How to Make a Web Application Using Flask in Python 3.” *DigitalOcean*, 17 Apr. 2020, www.digitalocean.com/community/tutorials/how-to-make-a-web-application-using-flask-in-python-3.

2) GeeksforGeeks. “Flask HTTP Methods, Handle GET & POST Requests.” *GeeksforGeeks*, 28 Jan. 2023, www.geeksforgeeks.org/python/flask-http-methods-handle-get-post-requests/.

3) GeeksforGeeks. “Generating Random Id’s Using UUID in Python.” *GeeksforGeeks*, 27 Jan. 2018, www.geeksforgeeks.org/python/generating-random-ids-using-uuid-python/.